Initial Approach

The goal was to classify network traffic captured over QUIC (HTTP/3, UDP port 443) into meaningful categories - video, audio, web, data - by capturing packets with tshark, decrypting them using TLS key logs from Chrome, and training an ML model on flow-level features.

The pipeline had three stages:

- **capture.py** - launches Chrome with SSL key logging, runs tshark capturing UDP/443 traffic for a fixed duration
- **decrypt.py** - uses tshark + the key log to extract HTTP/3 content-type headers per flow, maps them to labels, outputs CSVs
- **feature_extract.py** - reads raw pcaps, groups packets into flows by (src_ip, dst_ip), computes statistical features (packet count, bytes, IATs, burst count, upload ratio), joins with labels, produces training/test CSVs

Why the Initial Approach Failed

Problem 1 - Wrong/Missing content-type labels The original CONTENT_TYPE_MAP only had standard MIME types like video/, audio/, text/html. But YouTube doesn't send video/mp4 - it sends application/vnd.yt-ump, a proprietary multiplexed format. This meant almost all YouTube traffic fell through the label filter and was silently dropped. Result: ~200 rows, almost entirely web and data, almost no video or audio.

Problem 2 - CDN IP rotation Google's CDN and Akamai rotate IPs aggressively - sometimes mid-session. So a flow that started at IP X might continue from IP Y. Flows keyed by (src_ip, dst_ip) broke across these rotations, splitting what should be one logical stream into multiple orphaned fragments.

Problem 3 - Directional flow splitting (172.17.x.x → 142.250.x.x) and (142.250.x.x → 172.17.x.x) were treated as two separate flows, but they're two directions of the same connection. This halved the packets per flow, making features weaker and making many flows fail the minimum packet threshold.

Problem 4 - No time windowing (session-level averaging) All packets for an IP pair across the entire 60-second session were collapsed into a single row. A flow that bursted for 3 seconds then went idle produced one row with misleadingly low bytes_per_sec (total bytes ÷ 60s). The feature lost its signal entirely.

Problem 5 - Audio was fundamentally uncapturable Spotify uses TCP, not QUIC. SoundCloud uses HLS over TCP. The capture filter udp port 443 missed all audio streaming entirely. Even many SoundCloud sessions produced audio=1.

Fix 1 - Expanded CONTENT_TYPE_MAP + YouTube-specific types Added application/vnd.yt-ump → video, application/dash+xml → video, application/vnd.apple.mpegurl → video, audio/ogg, audio/mp4, video/mp2t (Spotify Canvas), application/protobuf → data (Spotify API), application/grpc → data (Google services), and others. This directly addressed the YouTube blind spot.

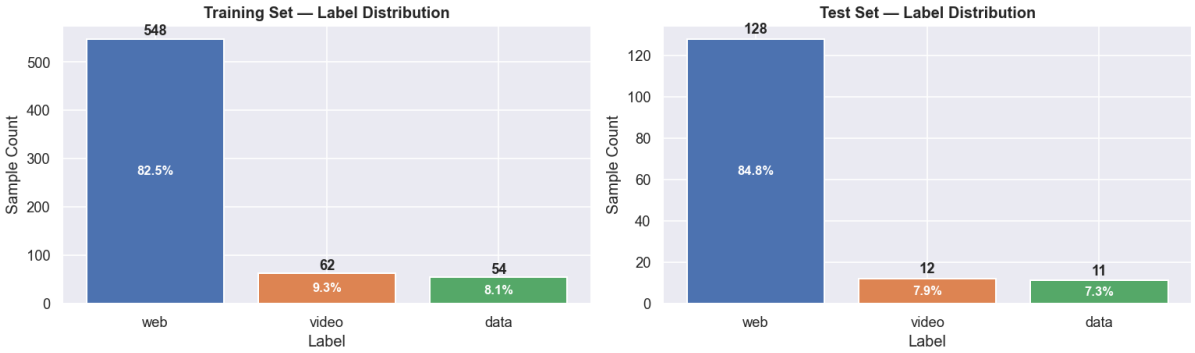
Fix 2 - SNI-based fallback labeling (second pass in decrypt.py) Even without decryption, TLS SNI is sent in plaintext during the handshake. A second tshark pass extracts tls.handshake.extensions_server_name and maps hostnames to labels - googlevideo.com → video, scdn.co → audio, sndcdn.com → audio, etc. This survives CDN IP rotation because the hostname is stable even when the IP isn't. Content-type labels take priority; SNI is only used as fallback.

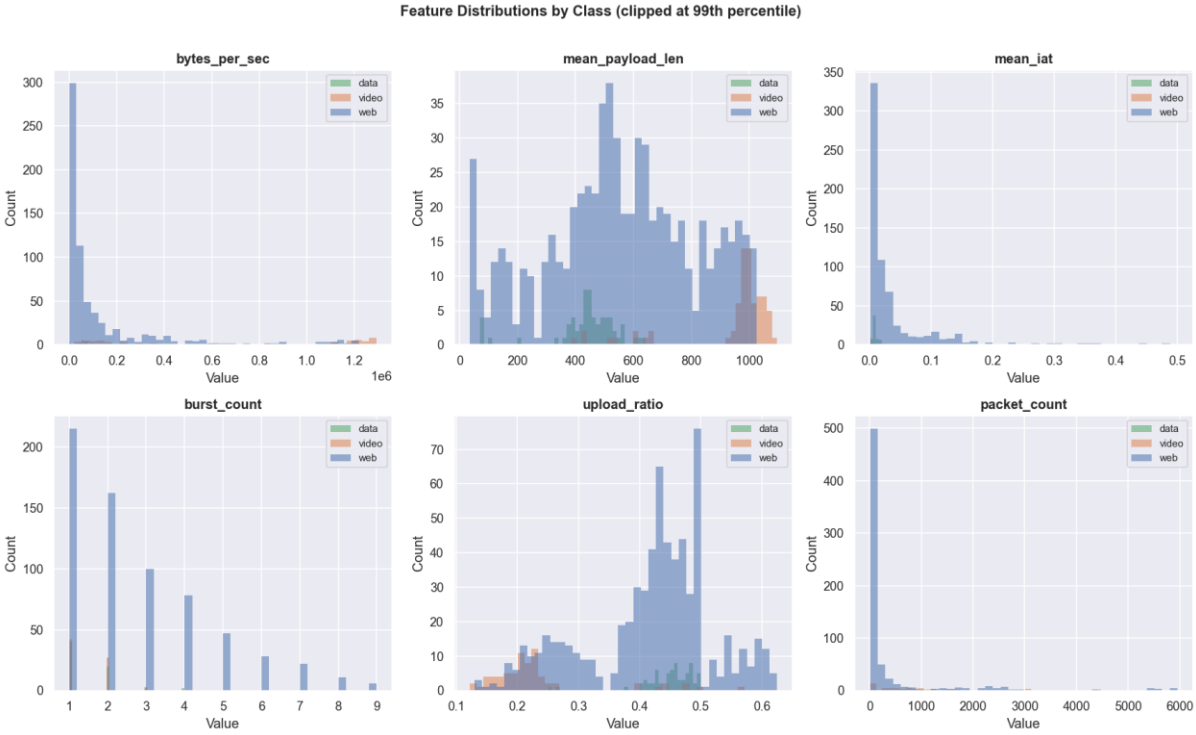
Fix 3 - Bidirectional flow unification Flow keys switched from ordered (src_ip, dst_ip) to tuple(sorted([ip_a, ip_b])). Both directions of a connection now merge into one flow, doubling packets per flow and making features much more representative.

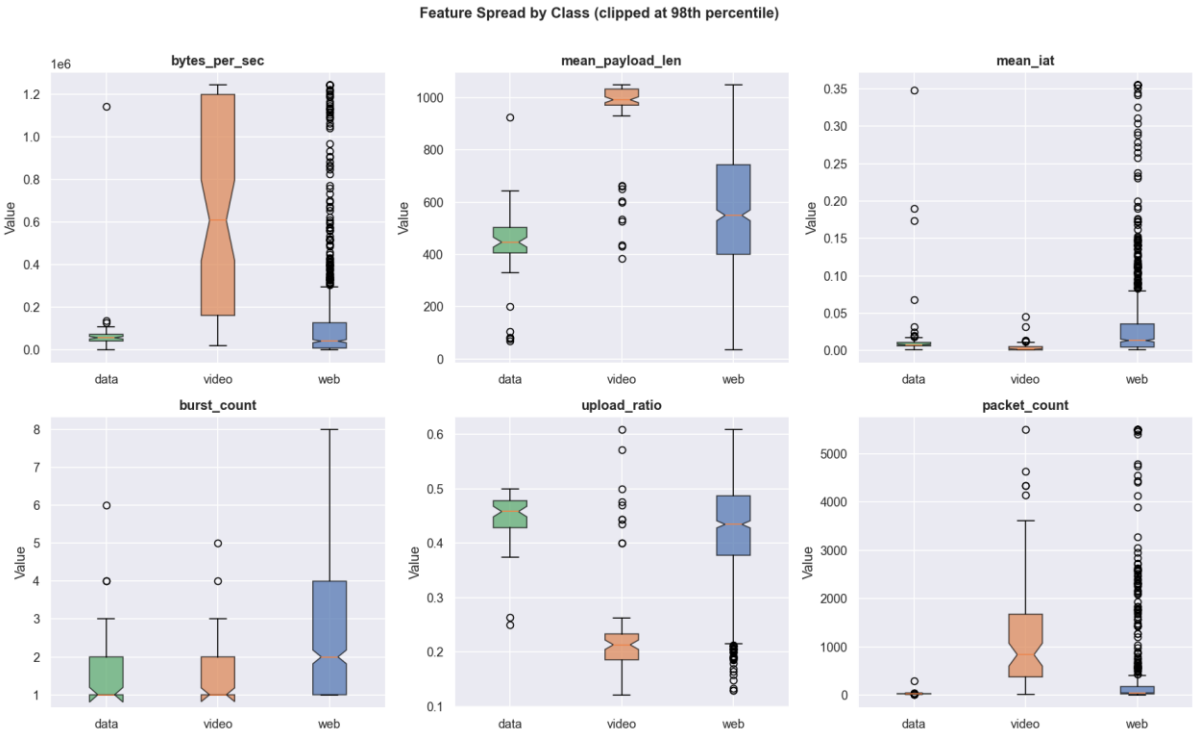
Fix 4 - Time-windowed feature extraction Instead of one row per flow per session, each flow is now sliced into 5-second windows. Windows with fewer than 5 packets are dropped (idle periods). Each window becomes one training row. This means bytes_per_sec within a window reflects an actual active burst, not a session average diluted by silence.

Fix 5 - Class weighting instead of SMOTE For the model, audio was dropped entirely (1 sample is not a class). Rather than SMOTE (which can synthesize unrealistic feature combinations for structured data), XGBoost's built-in sample weighting was used: weight[i] = total / (n_classes × count[i]).

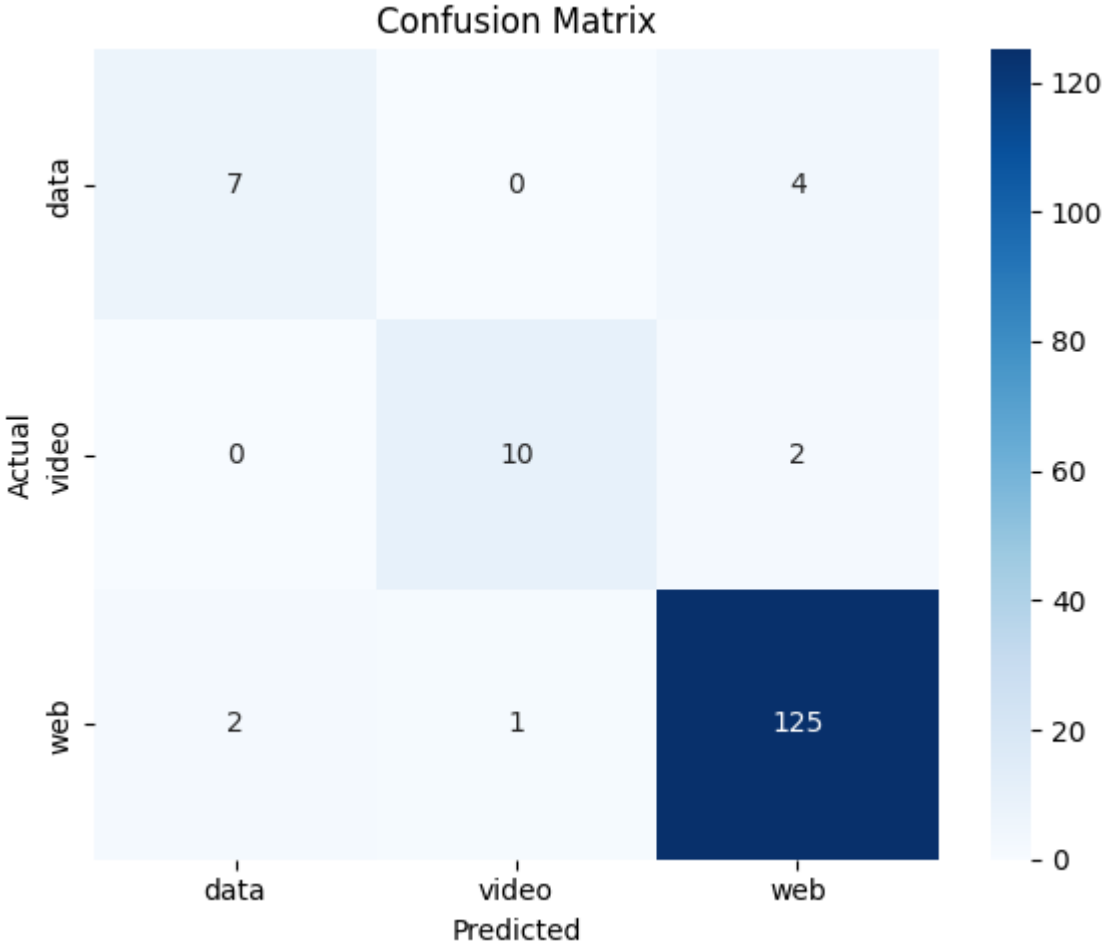
After all fixes: training data went from ~200 rows (mostly web) to 665 rows with web=548, video=62, data=54, audio=1 - real improvement on video, but audio remained a fundamental capture-layer problem due to TCP vs QUIC.


Training Set — Label Distribution
Sample Count
Label
web
video
data
548
82.5%
62
9.3%
54
8.1%
Test Set — Label Distribution
Sample Count
Label
web
video
data
128
84.8%
12
7.9%
11
7.3%


Feature Distributions by Class (clipped at 99th percentile)
bytes_per_sec
mean_payload_len
mean_iat
burst_count
upload_ratio
packet_count
Count
Value
1e6
data
video
web



Feature	Importance
mean_payload_len	0.169050
burst_count	0.152681
packet_count	0.075604
max_payload_len	0.074610
upload_ratio	0.066027
min_iat	0.063463
duration_s	0.062831
total_bytes	0.057474
min_payload_len	0.057102
mean_iat	0.056890
std_payload_len	0.048472
max_iat	0.043929
bytes_per_sec	0.043687
std_iat	0.028179


Confusion Matrix
data
video
web
Actual
7
0
4
0
10
2
2
1
125
data
video
web
Predicted
120
100
80
60
40
20
0